

Inteligência Artificial



Roteiro

1. Descrição do Problema
2. Modelagem do mundo
3. Framework e API Mínima
4. Implementação de um Agente Aleatório
5. Implementação do Dijkstra
6. Trabalho
 - a. Busca sem Informação
 - i. BFS, DFS, UCS
 - b. Busca com Informação
 - i. Greedy, A*
7. Entrega e Relatório

1. Descrição do Problema

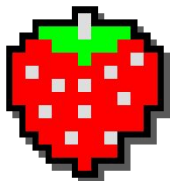
Pac-Man Labirinto

- O Labirinto do **Pac-Man** descreve a narrativa de um agente que **explora um ambiente inicialmente desconhecido** em busca do **morango**;
- Nesse labirinto, as **paredes e a névoa de guerra** representam os obstáculos que restringem a movimentação do **Pac-Man**;
- Entretanto, o **Pac-Man** conta com a ajuda dos fantasmas **Blinky**, **Clyde**, **Inky**, **Pinky** e o **Blue** para explorar o labirinto e encontrar o **morango**;
- O **objetivo final** da atividade é encontrar o **morango** no fim do labirinto;

Pac-Man Labirinto



- Nesse versão do jogo, os personagens são:
 - **Pac-Man** como o **agente explorador**;
 - **Morango** como o **objetivo final**;
 - Os **fantasmas** que representam a **fronteira de exploração**;
- Assim, o objetivo do **Pac-Man** é encontrar o **Morango**;
- **Condições de sucesso e fracasso:**



SUCESSO

Encontrar e alcançar
o morango

FRACASSO

- Ficar preso no Labirinto
- Não alcançar o morango

2. Modelagem do mundo

O Mundo

- O mundo é representado por um **grid bidimensional**.
- Cada célula do grid pode representar:
 - caminho livre;
 - parede;
 - saída;
 - célula ainda desconhecida pelo agente.
- O agente observa apenas o mapa conhecido até o momento.
- O agente sabe a posição final desejada (**objetivo**);
- Existem vários caminhos até o **objetivo**;
- A cada movimento, o ambiente revela células próximas do **fantasma** removido da **fronteira de busca**.

O Mundo

- O mundo é descrito como um grid **NxN**, em que **N** é o número de linhas e colunas;

	1	2	3	4
1				
2				
3				
4				

O Mundo

- Cada célula pode receber o valor:
 - UNKNOWN = -1
 - FREE = 0
 - WALL = 1
 - EXIT = 2

	1	2	3	4
1				
2	0			
3	 = 0			
4				 = 2

Ações relevantes

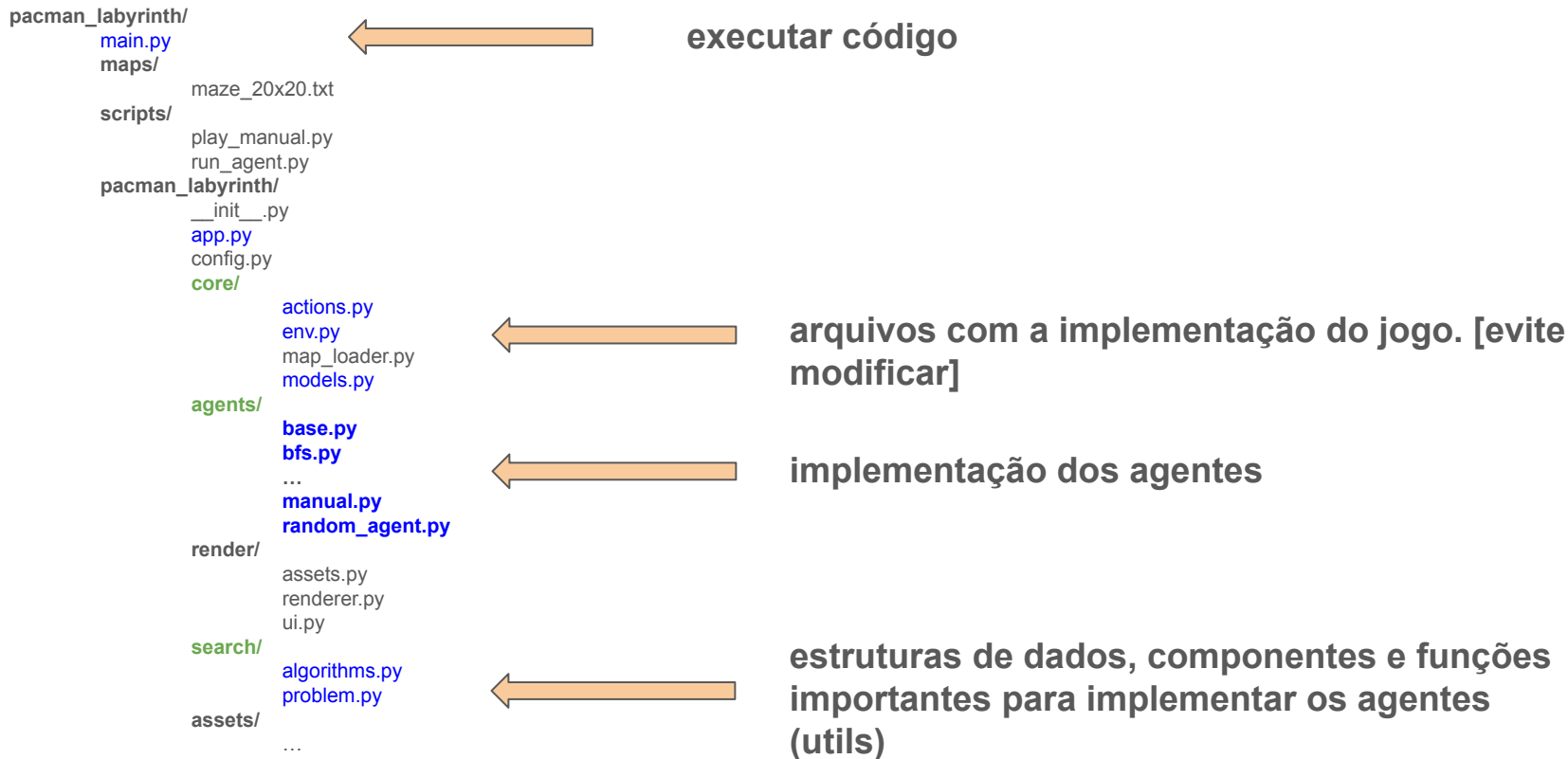
Ícone	Ação	Percepção
-	MOVE_*	Move o Pac-Man.
-	WAIT	Não altera posição; útil para depuração e políticas simples.

3. Framework e API Mínima

API do Jogo

- A API do jogo é simples; isso permite focar nas decisões de projeto do agente.

Estrutura do Projeto



Classes Importantes

Classe	Descrição	Localização
BaseAgent	Classe Abstrata para o agente; expõe act(...) e, opcionalmente, reset().	./pacman_labyrinth/agents/base.py
SearchAgentBase	Classe Abstrata para o agente de busca; (agente responsável apenas por rodar o algoritmo de busca). Expõe a função search(...)	./pacman_labyrinth/agents/search_agent_base.py
Percept	Agrupar posição, direção, visitados e outros recursos.	./pacman_labyrinth/core/models.py
Action	Enum de ações disponíveis no ambiente.	./pacman_labyrinth/core/actions.py
MazeEnv	Mantém o estado real do mundo e aplica env.step(action).	./pacman_labyrinth/core/env.py

Classes Importantes

Classe	Descrição	Localização
GridPlanningProblem	Representação do Problema de Busca em Grid	./pacman_labyrinth/search/problems.py
SearchResult	Resultado do algoritmo de Busca	./pacman_labyrinth/search/algorithms.py
FIFOFrontier	Implementa uma FIFO para fronteira de exploração	./pacman_labyrinth/search/algorithms.py
LIFOFrontier	Implementa uma LIFO para fronteira de exploração	./pacman_labyrinth/search/algorithms.py
PriorityFrontier	Implementa uma Fila de Prioridade para fronteira de exploração	./pacman_labyrinth/search/algorithms.py

Funções Importantes

Classe	Descrição	Localização
reconstruct_path	Reconstrói o caminho do Início ao Final dado um dicionário com os pais	./pacman_labyrinth/search/algorithms.py
GridPlanningProblem	Implementa várias funções importantes como o teste de objetivo, heurística, sucessores, etc.	./pacman_labyrinth/search/problems.py

Template Mínimo para um Agente

```
1 from base import BaseAgent
2 from ..core.actions import Action
3 from ..core.models import Percept
4
5 class MeuAgente (BaseAgent):
6     def __init__(self, seed: int | None = None):
7         pass
8
9     def reset(self):
10         return None
11
12     def act(self, percept, legal_actions):
13         return Action.WAIT
```



Template Mínimo para um Agente de Busca

```
1 from .search_agent_base import BaseAgent
2 from ..search.algorithms import SearchResult
3 from ..search.algorithms import reconstruct_path
4 from ..search.problems import GridPlanningProblem
5
6 class MeuAgenteDeBusca (SearchAgentBase):
7
8     def search(self, problem:GridPlanningProblem)
9         -> SearchResult:
10             return problem.solution([], [], 0.0, True)
```

É o que vocês
devem
implementar
no trabalho

O framework - onde colocar e como executar

1) **Passo 1:** editar o arquivo

- a) Editar o arquivo do algoritmo de busca em **pacman_labyrinth/agents/**.
- b) Exemplos: bfs.py e [dfs.py](#).

2) **Passo 2:** editar o arquivo do problema:

- a) Editar o arquivo para implementar funções como a heurística:
./pacman_labyrinth/search/problems.py

3) **Passo 2:** demonstrar no app

- a) Executar **main.py** para visualizar o comportamento no tabuleiro.
 - i) **\$ python3 main.py**
- b) No menu, escolher o agente que quer testar e o tamanho do mapa.

4. Implementação de um Agente Aleatório

Implementação de um Agente Aleatório

- Implementação de um agente baseline bem simples.
- Estratégia:
 - escolhe um movimento possível aleatoriamente;

Implementação de um Agente Aleatório

```
1 class RandomAgent(BaseAgent):
2     def __init__(self, seed: int | None = None):
3         self.rng = random.Random(seed)
4         print("Agente Aleatório!")
5
6     def act(self, percept: Percept, legal_actions: list[Action]) -> Action:
7
8         options = [
9             Action.MOVE_UP,
10            Action.MOVE_RIGHT,
11            Action.MOVE_DOWN,
12            Action.MOVE_LEFT,
13            Action.WAIT,
14        ]
15        return self.rng.choice(options)
```

5. Implementação do Dijkstra

Implementação do Dijkstra

- Implementação de um agente de busca usando o algoritmo de Dijkstra.
- Estratégia:
 - o algoritmo funciona construindo iterativamente uma "árvore de caminhos mínimos", expandindo a partir do ponto inicial para o próximo vértice mais próximo que ainda não foi totalmente processado;

Implementação do Dijkstra

```
1. from __future__ import annotations
2. from .search_agent_base import SearchAgentBase
3. from ..search.algorithms import SearchResult, reconstruct_path
4.
5.
6. class DijkstraAgent(SearchAgentBase):
7.     algorithm_name = "dijkstra"
8.
9.     def search(self, problem) -> SearchResult:
10.         frontier = problem.priority_frontier()
11.         frontier.push(0.0, problem.start)
12.
13.         start_key = problem.start.as_tuple()
14.         parents = {start_key: (None, None)}
15.         dist = {start_key: 0.0}
16.         expanded_set: set[tuple[int, int]] = set()
17.
18.         while frontier:
19.             priority, cur = frontier.pop()
20.             cost = priority[0]
21.             cur_key = cur.as_tuple()
```

```
22.             if cost > dist[cur_key]:
23.                 continue
24.             if cur_key in expanded_set:
25.                 continue
26.
27.             if problem.is_goal(cur):
28.                 actions, path = reconstruct_path(parents, cur)
29.                 return problem.solution(actions, path, cost, True)
30.
31.             expanded_set.add(cur_key)
32.             for action, nxt, step_cost in problem.successors(cur):
33.                 nxt_key = nxt.as_tuple()
34.                 new_cost = cost + step_cost
35.                 if new_cost < dist.get(nxt_key, float("inf")):
36.                     dist[nxt_key] = new_cost
37.                     parents[nxt_key] = (cur_key, action)
38.                     frontier.push(new_cost, nxt)
39.
40.         return problem.failure()
```

6. Trabalho

Exercício A

- Implemente os Algoritmos de busca sem informação para resolver o **Pac-Man Labirinto**;
 - BFS (Busca em Largura);
 - DFS (Busca em Profundidade);
 - UCS (Busca com Custo Uniforme);

Exercício B

- Implemente os Algoritmos de busca com informação para resolver o **Pac-Man Labirinto**;
 - Greedy (Busca Gulosa);
 - A* (A-estrela);

7. Entrega e Relatório

Entrega e checklist final

- Entregas:
 - Todo o projeto compactado com os arquivos dos algoritmos de busca.
 - Relatório curto em PDF.
- Condições do trabalho:
 - Trabalho individual.
 - Prazo de 3 semanas a partir da divulgação do trabalho no Classroom.
 - Entrega pelo Google Classroom.
 - Código deve funcionar no framework fornecido.
- Critérios observáveis:
 - Correção funcional dos agentes.
 - Coerência entre teoria e implementação.
 - Clareza do código.

Relatório

Seção	O que descrever
1. Identificação	Nome, RA e data
2. Visão Geral do Problema	Objetivo do Pac-Man Labirinto
3. Algoritmos de Busca sem Informação	Exercício A
4. Algoritmos de Busca com Informação	Exercício B
5. Testes	Como testou e quais diferenças observou entre os agentes
6. Declaração de uso de IA	Declarar se usou e como usou IA no desenvolvimento do trabalho
7. Melhorias	Limitações, extensões futuras e feedback opcional do framework e projeto (ajudará as próximas turmas)